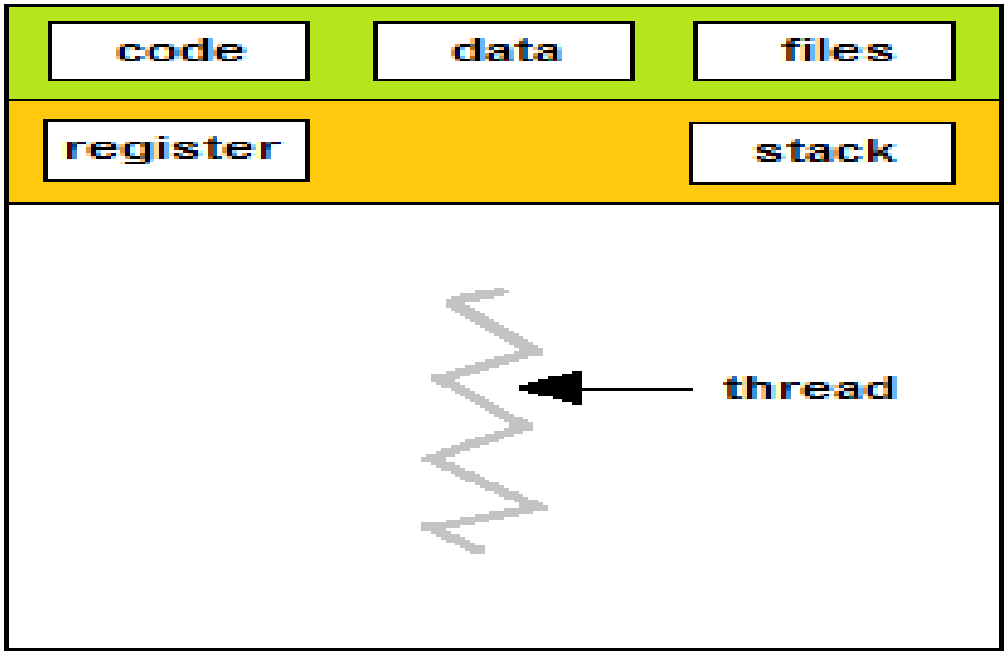
Threads

- A thread is a light weight process.
- A thread is a basic unit of CPU utilization, it comprises ;
  - Thread ID
  - Program counter
  - Register set
  - Stack

Single Threaded and Multi Threaded Process

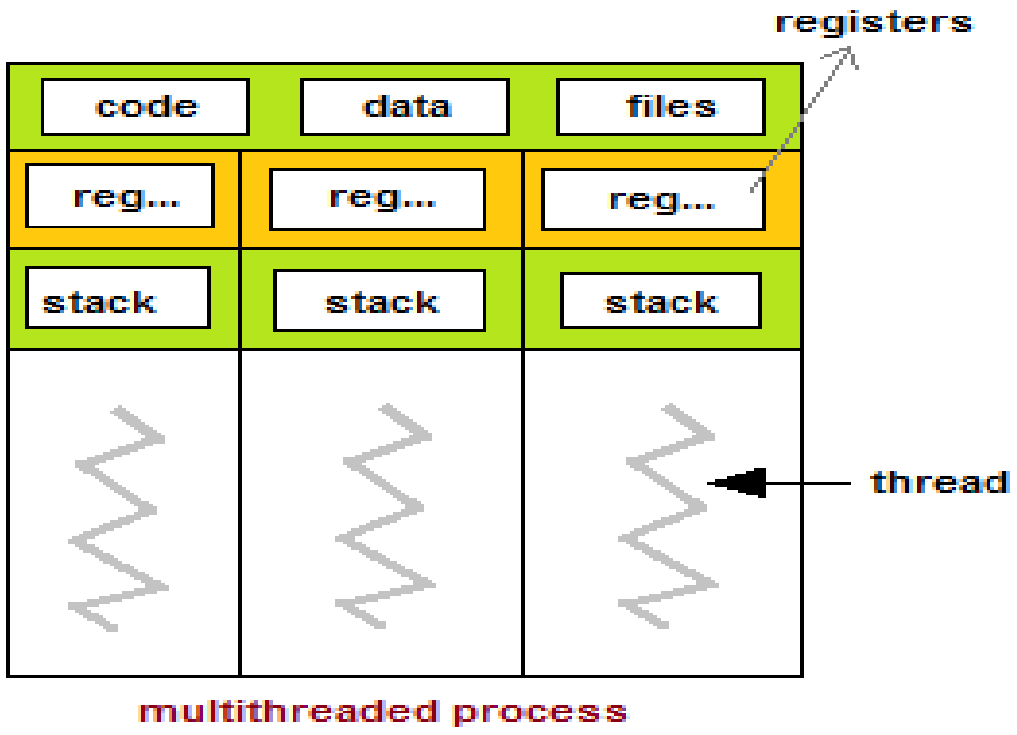
- Many software packages that run on modern desktop PCs are multithreaded. An application typically is implemented as a separate process with several threads of control.
- For example, a word processor ;
  - A thread for displaying graphics.
  - Another thread for responding to keystrokes from the user.
  - And third thread for performing spelling and grammar checking in the background.

Single Threaded Process



single-threaded process

Multi Threaded Process



Benefits of Multithreaded Programming

- There are five major benefits of multithreaded programming.
  - Responsiveness.
  - Resource sharing
  - Economy
  - Scalability
  - Less communication overheads

Responsiveness

- Multithreading increase the responsiveness to the user. Multithreading may allow a program to continue running even if part of it is blocked or is performing a lengthy operation.
- For instance, a multithreaded web browser could allow user interaction in one thread while an image was being loaded in another thread.

Resource Sharing

- Threads share the memory and the resources of the process to which they belong by default.
- The benefit of sharing code and data is that it allows an application to have several different threads of activity within the same address space.

Economy

- Allocating memory and resources for process creation is costly.
- Thread share the resources of the process to which they belong, so it is more economical to create thread.
- It is much more time consuming to create and manage process than thread.
- For e.g Solaris, creating a process is about thirty times slower than is creating a thread.

Scalability

- The benefits of multithreading can be greatly increased in a multiprocessor architecture, where threads may be running in parallel on different processors.
- A single threaded process can only run on one processor, regardless how many are available.

Less Communication Overheads

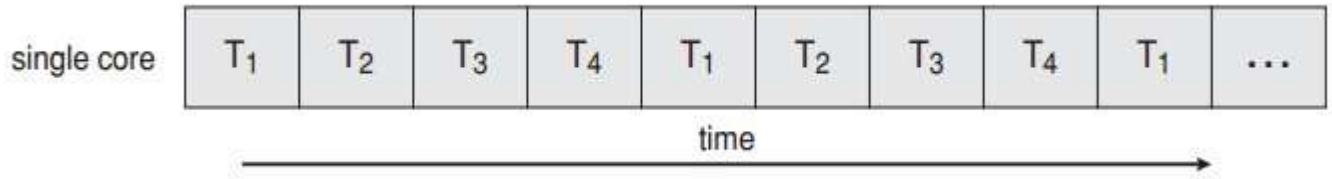
- Communicating between the threads of one process is simple because the threads share everything, address space.
- So, data produced by one thread is immediately available to all the other threads.

Single Core and Multi Core System

- A recent trend in system design has been to place multiple computing cores on a single chip, where each core appears as a separate processor to the operating system.
- Multithreaded programming provides the mechanism for more efficient use of multi-cores and improved concurrency.

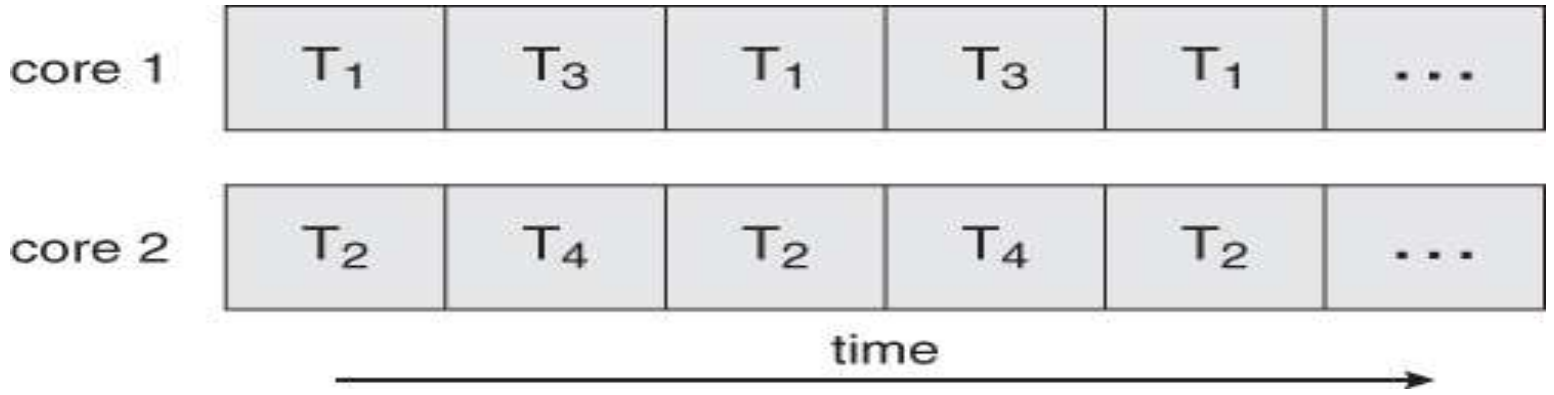
Thread Execution on Single Core System

- Consider an application with four threads (T1, T2, T3 and T4). On a system with a single computing core, concurrency merely means that the execution of a threads will be interleaved over time, as the processing core is capable of executing only one thread at a time. As shown in figure below;



Thread Execution on Single Multiple Core System

- On a system with multiple cores, however , concurrency means that the thread can run in parallel, as the system can assign a separate thread to each core.



Thread Library

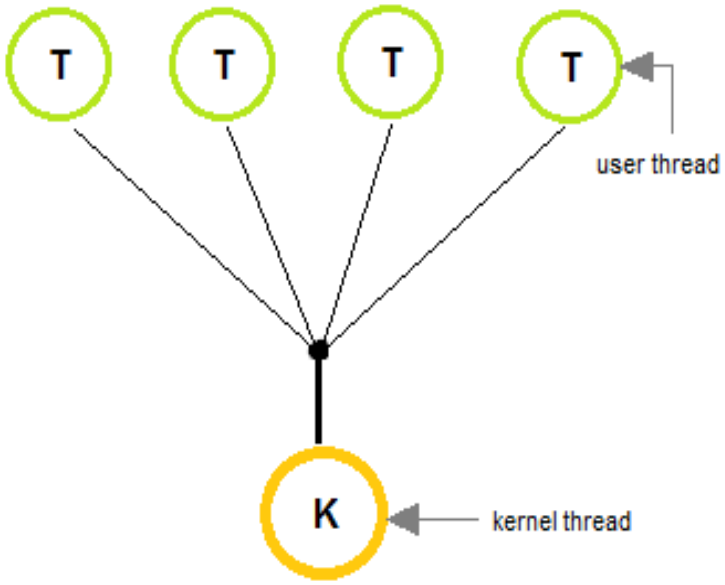
- A thread library provides the programmer with an Application programming interface (API) for creating and managing threads.
- Thread library contain code for;
  - Creating and destroying threads.
  - Passing messages and data between threads
  - Scheduling thread executing
  - Saving and restoring thread contexts

Kernel Threads

- Kernel threads exist within the context of a process and provide the operating system the means to address and execute smaller segments of the process.
- It also enables programs to take advantage of capabilities provided by the hardware for concurrent and parallel processing.
- Kernel threads are supported and managed directly by the operating system.

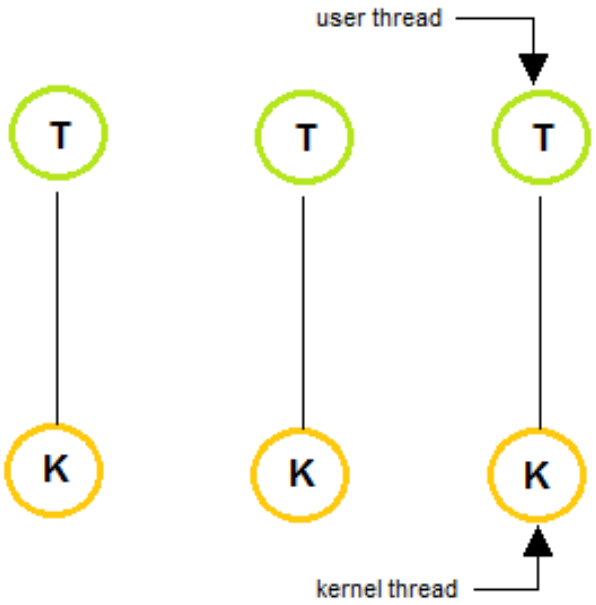
Multithreading Models (Many-To-One Model)

- In many-to-one model, many user level threads are map to one kernel thread.
- Thread management is done by the thread library in user space.
- This model is efficient , but the entire process will block if a thread makes a blocking system call.
- Only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multiprocessors.



Multithreading Models (One-To-One Model)

- In one-to-one model, each user thread map to kernel thread.
- It provides more concurrency than many-to-one model by allowing another thread to run when a thread makes a blocking system call.
- This model allows running multiple threads on multiprocessor system.



Multithreading Models (Many-To-Many Model)

- The many-to-many multiplexes many user level threads to a smaller or equal number of kernel threads.
- In many-to-many model, when a thread performs a blocking system call, the kernel can schedule another thread for execution.